

The Flux Sync Engine

Content-Addressed, Verify-Don't-Trust Block Backfill
for Gossip-Only DagKnight Chains

SIGIL / Flux Engineering
Rocky (Claude Opus) — agentic engineering log

June 2026 **v0.22**

Abstract

Gossip-based block propagation has a structural blind spot: a node only ever receives blocks produced *after* it joins the mesh. It cannot recover history. The **Flux Sync Engine** closes this gap for both the Flux supercluster and the SIGIL chain with two cooperating layers. The lower layer, **flux-sync**, makes every block *content-addressed* through the **flux-aether** erasure store — a BLAKE3 root over K -of- N erasure-coded, encrypted shards — so that a retrieved block is rejected unless its bytes hash back to the advertised root (*verify-don't-trust*). Peers exchange a compact manifest of (**height**, **content_root**) pairs and compute the exact set of blocks they lack. The upper layer, the **block_sync** pipeline, drives this over a live libp2p mesh: a frontier-anchored, pipelined sliding-window backfill that streams up to twelve in-flight chunk requests across a rotating, self-benching peer pool, learns the network tip from a signed out-of-band oracle, and — for light “monitor” nodes — snaps its sync base to a recent window so it reaches the head in seconds rather than crawling six million blocks from genesis. A split hot/cold loop keeps a cryptographic spine-verification watermark honest without ever gating the ingest path. We describe the data structures, the concurrency model, the security argument, and the specific failure modes each mechanism was built to defeat.

Contents

1	The Gap	1
2	Design Goals	2
3	Layer 1 — The Content-Addressed Backfill Engine	2
3.1	Content addressing via flux-aether	2
3.2	The manifest and the missing-set	2
3.3	The store: ingest, serve, import, retrieve	3
3.4	The verify-don't-trust property	3
4	Layer 2 — The Live Sync Pipeline	4
4.1	Wire protocol	4
4.2	The pipelined sliding window	4
4.3	Frontier-anchored refill	4
4.4	Peer health and the healthy-peer floor	5
4.5	The pull height-probe: breaking the gossip $\leftrightarrow$ backfill deadlock	5
4.6	The tip oracle	5
4.7	Monitor fast-snap	5
4.8	The post-snap frontier recompute (v0.21.1)	6

4.9	Dynamic base: self-healing around pruned history	6
4.10	The split hot/cold loop	6
5	Failure Modes Defeated	7
6	Security Analysis	7
7	Performance	8
8	Conclusion	8

1 The Gap

`flux-p2p` is a gossipsub mesh. When a producer mints a block it is flooded to every subscriber of the `/sigil/g0/blocks` topic. This is excellent for *liveness* — a connected node sees new blocks within a gossip round — but it offers **no history**. A fresh node, or one that was offline, collects only the blocks minted from the moment it subscribes. There is no request/response primitive for “give me blocks $[a, b]$ ”. The SIGIL chain hit this on 2026-06-08: a newly launched cockpit node sat on the live tip with a seven-million-block hole beneath it and no way to fill it.

A naive fix — “ask a peer for block h and trust the reply” — reintroduces the oldest attack in the book: a malicious or buggy peer can serve fabricated history. Because gossip carries no proof-of-retrieval, the receiver has nothing to check the bytes against. The engine therefore has two jobs, and they are separable:

1. **Addressability + integrity** (the `flux-sync` engine): name every block by a hash of its content, and make retrieval self-verifying.
2. **Scheduling + transport** (the `block_sync` pipeline): figure out *what* is missing, fetch it fast over a real WAN mesh of a handful of peers, and stay live under churn, pruning, and silence.

2 Design Goals

G1 — Verify-don’t-trust.

A block accepted from a peer must hash to the root the peer advertised. A lying peer cannot poison history; the worst it can do is waste a round-trip.

G2 — Exact backfill.

A node fetches precisely the blocks it lacks — no full-chain re-download, no redundant ranges — computed as a set difference over content roots.

G3 — No barrier stalls.

The single slowest or most-behind peer must never gate the whole cohort. Throughput tracks the *fastest* available peers, not the slowest.

G4 — Liveness without gossip.

A node that connects but receives no gossip (graft failure, silent producers) must still discover the tip and backfill. The engine cannot depend on inbound gossip to bootstrap.

G5 — Monitor-fast.

A light node hundreds of thousands of blocks behind must reach the live, *verified* tip in seconds, deferring full contiguous integrity to a constant-size fold proof rather than a 6M-block download.

G6 — Honest watermarks.

“Downloaded” and “cryptographically verified as one chain” are distinct states and are reported separately; verification never gates the ingest hot path.

3 Layer 1 — The Content-Addressed Backfill Engine

The lower layer is the crate `flux-sync`. It is the *engine* — store, manifest, and diff — testable entirely in-process. The wire layer that gossips manifests and ships shards lives one level up (Section 4).

3.1 Content addressing via flux-aether

Every block’s bytes are handed to `flux-aether`’s `shard_file`, which produces:

- a `FileBlock` carrying a 32-byte BLAKE3 `content_root` over the original bytes, and
- a vector of N erasure-coded, encrypted `Shards` (default 64 KiB stripe), of which any K reconstruct the block.

The pair persists as one file named for its own root:

```
<dir>/<content_root_hex>.block = bincode((FileBlock, Vec<Shard>)).
```

Naming the file after the verified content is the first half of verify-don’t-trust: the filename *is* a claim that can be checked.

3.2 The manifest and the missing-set

A node’s holdings are summarised by a `SyncManifest` — a list of `BlockRef{height, content_root}` with the highest height cached as `tip`. Two manifests yield a backfill work-list by set difference over roots:

```
1 pub fn missing(local: &SyncManifest, remote: &SyncManifest) -> Vec<BlockRef> {
2     let have = local.roots(); // HashSet<content_root>
3     let mut out: Vec<BlockRef> = remote.refs.iter()
4         .filter(|r| !have.contains(r.content_root.as_str()))
5         .cloned()
6         .collect();
7     out.sort_by_key(|r| r.height); // ascending: apply in order
8     out
9 }
```

The difference is taken over *content roots*, not heights. This is what makes it robust to reorganisations: if two nodes hold different blocks at the same height, the roots differ and both are surfaced; if they hold the same block, the roots match and it is correctly skipped (Goal G2). The result is sorted ascending so the caller applies it as a contiguous spine.

3.3 The store: ingest, serve, import, retrieve

`BlockStore` is a thin content-addressed layer over the aether shards. Four operations form the protocol surface:

```
1 // Local producer / re-ingest. Idempotent: identical bytes -> identical
2 // root -> dedup (no rewrite). Returns the content_root hex.
3 pub fn ingest(&self, bytes: &[u8]) -> io::Result<String>;
4
5 // The on-wire payload a peer sends on a ‘want’.
```

```

6 pub fn blob(&self, root_hex: &str) -> Option<Vec<u8>>;
7
8 // Import a blob received from a peer, keyed by its ADVERTISED root.
9 // Returns whether the stored bytes actually reassemble AND hash to
10 // root_hex; a mismatch is deleted on the spot (anti-poison).
11 pub fn import_blob(&self, root_hex: &str, blob: &[u8]) -> io::Result<bool>;
12
13 // Retrieve + BLAKE3-verify by content_root. None if absent or corrupt.
14 pub fn retrieve(&self, root_hex: &str) -> Option<Vec<u8>>;

```

`retrieve` is the verification chokepoint. It deserialises the `(FileBlock, Vec<Shard>)`, calls `reassemble`, which re-hashes the reconstructed output and asserts it equals `fb.content_root`, and then — belt and suspenders — asserts the *filename* matches the verified content. Either check failing yields `None`.

3.4 The verify-don't-trust property

Property 1. If `import_blob(root, blob)` returns `true`, then a subsequent `retrieve(root)` returns exactly the bytes *b* for which $\text{BLAKE3}(b) = \text{root}$.

Proof sketch. `import_blob` writes the blob and immediately calls `retrieve`, returning `true` only if retrieval succeeds. Retrieval succeeds only if `reassemble` reconstructs bytes whose BLAKE3 digest equals the stored `content_root` and the hex of that root equals the filename `root`. A tampered blob therefore fails: either it does not deserialise, or it reassembles to bytes whose digest $\neq$ `root`. On failure the bad file is deleted, so a poison attempt leaves no residue. $\square$

The end-to-end test exercises exactly this: node *B* holding blocks 1,2 computes `missing` against node *A*'s manifest `{1,2,3,4}`, fetches *A*'s blobs for 3,4, imports-with-verification, and confirms it is fully synced — then proves a garbage blob written under a *valid* root is rejected.

4 Layer 2 — The Live Sync Pipeline

The upper layer, `P2PBlockSync` in `sigil-top`, drives the engine over a real libp2p mesh. It is the component a user watches in the cockpit, and its evolution (v0.7 through v0.22) is a catalogue of the ways a small-peer WAN backfill can stall. The shape that survives is a **frontier-anchored, pipelined sliding-window backfill**.

4.1 Wire protocol

Backfill is a point-to-point request/response over the flux-p2p channel, byte-compatible with the producing node's server:

```

1 pub struct BackfillReq { pub from: u64, pub to: u64, pub headers_only: bool }
2 pub struct BackfillResp { pub blocks: Vec<serde_json::Value> }

```

With `headers_only = true` (v0.7.27) the responder replies with a compact 'H'-tagged bincode `(Vec<SigilBlock>)` — roughly $20\times$ less wire than full-block JSON, and no JSON to lex — because a light monitor stores only headers. Old nodes ignore the flag and reply with full-block JSON; the client detects the leading byte and falls back. Crucially, the responder **clamps** the served range to its own tip (`hi = req.to.min(top)`), which the client exploits below.

4.2 The pipelined sliding window

The naive design fired one request per peer then `join_all`-barriered on all of them with a timeout — so the slowest peer gated every cycle (Goal G3 violated). The engine replaces the barrier with a continuously-refilled pool of independent `tokio::spawn`'d requests whose results return on an `mpsc` channel:

```

1 const CHUNK: u64 = 4096; // 4 MB headers; halves tail latency
2 const MAX_INFLIGHT: usize = 12; // independent in-flight chunks
3 const REQ_TIMEOUT: Duration = Duration::from_secs(10);
4 const EMPTY_BENCH: Duration = Duration::from_secs(10); // re-route empty ranges
5 const BENCH: Duration = Duration::from_secs(4); // re-route timeouts

```

Up to twelve chunks stream in parallel; a slow peer never blocks the fast ones. Out-of-order arrival is fine: the store’s height index plus its contiguous `advance()` reorder chunks as they land — *the store is the reorder buffer*. The sync loop runs on a dedicated three-worker tokio runtime so the spawned request tasks make progress even while the main loop is busy draining the gossip flood.

4.3 Frontier-anchored refill

The scheduler is deliberately memoryless. Every cycle it (re)issues the next `MAX_INFLIGHT` consecutive chunks *starting at the current contiguous frontier*, skipping any height already in flight:

```

1 store.advance();
2 let frontier_chunk = ((store.synced_to() / CHUNK) * CHUNK).max(sync_base);
3 // ... after any base/snap change, RECOMPUTE (see v0.21.1 below) ...
4 for i in 0..(MAX_INFLIGHT as u64) {
5     if inflight >= MAX_INFLIGHT { break; }
6     let start = frontier_chunk + i * CHUNK;
7     if start >= peer_best { break; } // past the tip
8     if !assigned.insert(start) { continue; } // already in flight
9     let peer = healthy[rr % healthy.len()]; // rotate the peer pool
10    rr = rr.wrapping_add(1);
11    // spawn: timeout(REQ_TIMEOUT, net.send_request(peer, BackfillReq{..}))
12 }

```

There is no cursor that can race ahead of a stalled lead chunk, no separate retry queue. The frontier chunk — the only chunk that actually advances `synced_to` — is *always* in flight; a stuck chunk clears its `assigned` slot on completion or timeout and is re-issued to a rotated peer next cycle. This memoryless anchoring is the resolution of the v0.10.0 “frontier-stall” bug, where look-ahead machinery consumed all twelve slots with far-ahead ranges while the lead chunk starved.

4.4 Peer health and the healthy-peer floor

Each peer carries a *benched-until* timestamp. A timeout benches it for `BENCH`; an *empty* reply over a still-needed range benches it longer (`EMPTY_BENCH`) because empty-over-needed means the peer genuinely lacks that range (e.g. it pruned early history). But with only ≈ 4 peers, a burst of timeouts could bench the *entire* pool and collapse throughput to zero. The **healthy-peer floor** (v0.17) makes benching advisory: if every peer is benched, fall back to the full connected set so the probe and refill keep firing best-effort.

4.5 The pull height-probe: breaking the gossip $\rightleftharpoons$ backfill deadlock

The refill is gated on `peer_best > 0` — it must know a target tip. If `peer_best` were learnable only from inbound gossip, a node that grafts but receives no gossip would keep `peer_best = 0`, never fire a request, and sit at base forever — a deadlock (Goal G4 violated). The **pull height-probe** breaks it without any responder change:

```

1 // Ask a healthy peer for the OPEN-ENDED range [frontier, u64::MAX].
2 // The responder clamps to its own tip, so the reply's max header height
3 // is a real LOWER BOUND on that peer's head -- learnable without gossip.

```

```

4 let payload = BackfillReq { from: frontier_chunk, to: u64::MAX, headers_only: true };
5 // on reply: if let Some(maxh) = max_header_height(&b) { peer_best = peer_best.max(maxh) }

```

Each probe reply doubles as up to a chunk of free, verified headers. It is re-issued every 500 ms so `peer_best` tracks the head and the refill self-sustains all the way to the tip.

4.6 The tip oracle

There remains a subtle trap: the monitor's `/api/v1/status` endpoint mis-routed to a near-empty RPC daemon that returned `height = 2`, so `peer_best` was seeded at 2 and the fast-snap (gated on `peer_best >> synced`) never fired — a genesis crawl at ~ 1 blk/s. The fix (v0.17) is a signed, producer-published out-of-band oracle:

`https://sigilgraph.fluxapp.xyz/sigil-tip-live.json` $\rightarrow$ `{"height": ~6,900,000, ...}`

A **dedicated OS thread** (v0.21) polls it every 3s with a *blocking* HTTP client, isolated from the busy tokio runtime where the async fetch was non-deterministically starved, and seeds `peer_best` directly. Each request carries a millisecond cache-buster so no CDN can pin a stale tip.

4.7 Monitor fast-snap

A light monitor gains nothing from a contiguous genesis $\rightarrow$ tip crawl (Goal G5). Once the tip is known, it *snaps* its base to a recent window the producers serve fast:

```

1 const RECENT_WINDOW: u64 = 2_048; // pin the base ~1 chunk under the live tip
2 if recent_only && peer_best > RECENT_WINDOW {
3     let new_base = peer_best.saturating_sub(RECENT_WINDOW).max(sync_base);
4     if new_base > store.base() {
5         store.set_base(new_base);
6         store.advance();
7         assigned.clear(); // drop stale below-base in-flight requests
8         // snap fired -> base moved -> FRONTIER IS NOW STALE
9     }
10 }

```

The snap is driven *every tick* toward tip — `RECENT_WINDOW` (monotonic in the base), so a one-shot snap can never park $\sim 10k$ under a gappy final range and stick: as long as the tip oracle stays fresh, the base and the synced frontier chase the head and the monitor never falls behind.

4.8 The post-snap frontier recompute (v0.21.1)

The snap exposes an ordering hazard. `frontier_chunk` is computed near the top of the loop, *before* the snap moves the base. A snap that fires after that computation leaves the refill requesting genesis-area chunks (`start = 1,4097,...`) — heights now *below* the new base, which ingest but never advance `synced_to`. The monitor displays at the base yet reads 0.0% / 0 blk/s forever. The fix recomputes the frontier from the *current* (post-snap) `synced_to` immediately before the refill:

```

1 // v0.21.1: recompute AFTER the snap so the refill targets the snapped window.
2 let frontier_chunk = ((store.synced_to() / CHUNK) * CHUNK).max(sync_base);
3 if peer_best > 0 { /* ... refill from frontier_chunk ... */ }

```

4.9 Dynamic base: self-healing around pruned history

Producers prune early history from their RAM serving-window, and the disk range-serve of pruned-low ranges is unreliable. If the frontier chunk stays unservable by *all* peers for ≥ 2 s while

a strictly higher block has actually been received, the loop advances the base one chunk (full-sync) or jumps straight to the recent contiguous window (monitor), re-anchoring the verified spine at the lowest height the mesh *honestly* serves — not necessarily genesis. Critically, this gates on `best_height` (the true maximum block received), *not* the possibly-seeded `peer_best`, so at the mesh’s real serving ceiling the sync *holds* instead of thrashing the base toward an unreachable tip.

4.10 The split hot/cold loop

Verification reads the database and must never gate ingest (Goal G6). The loop is therefore split:

Fast periodic (150 ms).

`store.advance()` walks any newly contiguous heights, updates the dynamic base, and publishes the TUI/window state. Cheap; no db scan.

Slow periodic (1.5 s).

`verify_to(store, 60_000)` walks only the *new* contiguous headers — precheck plus parent-linkage back to the verified spine — persists the cryptographic watermark `verified`, records any genuine integrity break, and flushes the memtable to an SST so it cannot balloon during a multi-million-block sync.

`blocks_synced` (“downloaded”) and `verified` (“downloaded *and* validated as one chain back to the spine”) are reported as distinct watermarks. The full-sync completion gate watches `verified`, not `blocks_synced`. For a monitor, the displayed sync height is the *verified live tip* carried in `peer_best` (the signed tip-proof), because the newest blocks are not reliably range-served in real time and a contiguous frontier would always lag the head — an unwinnable race. The cryptographic spine watermark stays honest regardless of what the bar shows.

5 Failure Modes Defeated

The engine’s parameters are not arbitrary; each encodes a specific stall observed in production.

Version	Symptom	Mechanism
v0.9.5	<code>synced</code> stuck at 0; per-event gossip log starved the loop	Bounded gossip drain ($\leq 128/\text{iter}$); no per-event logging
v0.10.0	Frontier-stall: look-ahead consumed all slots with far-ahead ranges	Memoryless frontier-anchored refill
v0.10.0	current-thread runtime: request tasks only ran on <code>await</code>	Multi-thread (3-worker) runtime
v0.12.1	4 s timeout < WAN chunk time $\Rightarrow$ every request timed out	<code>REQ_TIMEOUT</code> raised, later tuned to 10 s
v0.15.1	8 KB chunks landed in bursts then stalled (2k $\rightarrow$ 2 blk/s swings)	<code>CHUNK = 4096</code> ; <code>MAX_INFLIGHT = 12</code>
v0.15.1	45 s empty-bench drained the 4-peer pool to ~ 0	<code>EMPTY_BENCH = 10 s</code> ; <code>BENCH = 4 s</code>
v0.17	Burst of timeouts benched the whole pool $\Rightarrow 0$ blk/s	Healthy-peer floor (bench is advisory)
v0.17	<code>/api/v1/status</code> returned height 2 $\Rightarrow$ snap never fired	Signed <code>sigil-tip-live.json</code> oracle
v0.18.5	Gossiped network tip clobbered <code>sync_height</code> (0 blk/s display)	Keep <code>sync_height</code> from <code>synced_to</code> ; tip only updates <code>peer_best</code>

Version	Symptom	Mechanism
v0.21	async tip-fetch starved by sync workload (<code>peer_best</code> froze)	Dedicated blocking tip-poller OS thread
v0.21.1	0 blk/s after snap: stale frontier requested below-base chunks	Recompute <code>frontier_chunk</code> post-snap
v0.21.1	“560k behind, 0 blk/s”: unservable 6M-block middle	Monitor base jumps straight to the recent window
v0.22	Monitor bar stuck “ N k behind” chasing un-range-served head	Display the verified live tip; keep verified honest

6 Security Analysis

History poisoning. Defeated by Property 1: a block is accepted only if its bytes hash to the advertised content root, so a peer cannot inject fabricated history. The cost of a poison attempt is one wasted round-trip and a deleted file.

Tip forgery. The network tip arrives by three independent paths — gossiped `SyncProgress`, the pull height-probe (clamped to the peer’s real served head), and the producer-signed `sigil-tip-live.json`. The fast-snap acts on `peer_best`, which only ever *rises*; a forged *high* tip merely makes the monitor request ranges no honest peer serves (an empty-bench, not a fork), while the dynamic-base logic gates progress on `best_height` — blocks actually received — so the displayed verified watermark cannot be inflated past what was cryptographically checked.

Eclipse / silence. A node that grafts to a silent or hostile mesh still discovers a usable tip via the pull-probe and the out-of-band oracle (Goal G4); it does not depend on any peer choosing to gossip. With no honest peer, no progress is made — but no *false* progress is reported either.

Shard confidentiality. Aether shards are encrypted under `FLUX_AETHER_KEY`; a passive observer of the wire sees ciphertext shards, and integrity rests on the BLAKE3 root rather than on transport trust.

7 Performance

The pipeline’s throughput ceiling is set by the apply path, not the scheduler. The verify budget of 60,000 headers per 1.5 s pass lifts the verified-watermark ceiling to ~ 40 k blk/s, above the ~ 33 k target and clear of the verify core’s measured 52k/s (chronos turbosync). Header-only chunks cut wire volume $\sim 20\times$ versus full-block JSON; 4 MB chunks with twelve-deep pipelining keep tail latency low enough that a single stuck slot frees within one `REQ_TIMEOUT` without cratering the aggregate rate. For a light monitor the relevant metric is not contiguous throughput at all: the fast-snap reaches the verified live tip in *seconds*, and full-chain integrity is discharged by a constant-size fold proof rather than a multi-million block contiguous download.

8 Conclusion

The Flux Sync Engine restores history to a gossip-only chain without surrendering the integrity guarantees gossip lacks. The lower layer reduces “did this peer tell the truth?” to a single hash check by naming every block after its content; the upper layer turns “fetch what I’m missing” into a memoryless, frontier-anchored, self-healing pipeline that stays live under churn, pruning,

and silence, and lets a light node reach the verified head in seconds. The design’s parameters are a fossil record of real stalls; its invariants — verify on retrieve, anchor at the frontier, raise-only tips, separate downloaded from verified — are what keep it honest. The same engine serves both the Flux supercluster and the SIGIL chain, which is the point: the gap it closes was never chain-specific.

Source of record: `flux/crates/flux-sync/src/lib.rs` (engine) and `sigil/crates/sigil-top/src/block_sync.rs` (pipeline). Version tags throughout cite the commit that introduced each mechanism.